实验二　可综合组合逻辑电路实验

一、实验目的

1.　掌握可综合 Verilog 语言进行组合逻辑设计的使用；

2.　学习测试模块的编写、综合和不同层次的仿真。

二、实验内容

1. 设计一款可综合的 2 选 1 多路选择器，并编写 testbench 测试

2. 设计一款可综合的 2-4 译码器，并编写 testbench 测试

3. 设计一款可综合的 8-3 编码器，并编写 testbench 测试

4. 设计一款可综合的 1 位二进制比较器，并编写 testbench 测试

5. 设计一款可综合的 2+2 位简单全加器，并编写 testbench 测试

6. 自学 verilog 编程规范和面向硬件电路的设计思想，小结与高级语言编程的主要区别。

三、实验步骤

1. 建立工程文件，编写模块源码和测试模块,要求测试模块能对源文件进行比较全面的测试。

2. 编译源码和测试模块，用测试模块对源文件进行测试，并综合仿真。得到波形图。

3. 记录数据并完成实验报告。

四、课上学习

1. Verilog 编程规范

系统介绍了 Verilog 编程的规范要求，强调良好编程风格对减少硬件资源消耗、提高设计频率及可维护性的重要性。命名规则部分明确了字符集、大小写、信号与模块命名习惯等，确保命名清晰易懂且唯一。注释规范涵盖文件注释、模块注释的模板与要求，便于代码理解和版本追溯。代码格式方面，对端口申明、内部连线排布、缩进对齐、语句书写等作出详细规定，如每行一个语句、避免硬编码数值、使用参数定义常数等。

2. EDA

围绕 EDA 技术与 Verilog 设计展开，先概述 EDA 技术的定义、发展历程及应用范畴，对比了 Top-down 与 Bottom-up 两种设计方法，突出 Top-down 的高效性。介绍了 IP 核的分类（软 IP、固 IP、硬 IP）及 SOC 设计相关概念。详细讲解了 EDA 设计流程，包括设计输入（原理图、HDL 文本）、综合（行为、逻辑、版图综合）、适配、仿真（功能、时序仿真）和编程下载等关键环节。还列举了常用 EDA 软件工具，涵盖集成开发、逻辑综合、仿真工具等，并分析了 EDA 技术向高集成度、高自动化、智能化发展的趋势，为理解 EDA 技术体系和应用场景提供了全面指引。

3. 面向硬件思想

聚焦基于 Xilinx 芯片的 HDL 语言高级进阶，核心围绕面向硬件电路的设计思维展开。重点讲解了阻塞赋值与非阻塞赋值的区别及适用场景，明确组合逻辑用阻塞赋值、时序逻辑用非阻塞赋值的原则。还阐述了 Verilog 程序的并行执行特点、可综合性（Synplify Pro 支持与不支持的结构）、面积与速度的权衡方法。同步电路设计原则、模块划分规范（信息隐蔽、资源复用等）以及通用代码风格（逻辑复用与复制、if 和 case 语句使用、避免意外锁存器）也是核心内容，通过实例和代码片段帮助理解硬件设计的关键逻辑与实操要点。

4.与高级编程语言的区别

Verilog 是一种硬件描述语言（HDL），与高级编程语言如 C++或 Java 有本质区别。Verilog 主要用于设计和模拟数字电路，支持并行处理和时间建模，代码可被综合成实际硬件。而高级语言主要用于编写顺序执行的软件程序，抽象层次更高，不直接处理硬件。Verilog 的语法和结构专为描述硬件模块和连接设计，使用 EDA 工具进行综合和仿真。高级语言则用编译器和调试器等工具，侧重算法和数据结构。简言之，Verilog 面向硬件设计，高级语言面向软件开发。

五、实验报告

1. 设计一款可综合的 2 选 1 多路选择器，并编写 testbench 测试

模块源码：

```
module test (
    input  wire a,      // 输入 0
    input  wire b,      // 输入 1
    input  wire sel,    // 选择信号
    output wire y       // 输出
);

assign y = sel ? b : a;

endmodule
```

测试模块：

```
`timescale 1ns/1ps
 module test_tb;

    reg a;
    reg b;
    reg sel;
    wire y;

    // 实例化待测模块
    test uut (
        .a(a),
        .b(b),
        .sel(sel),
        .y(y)
    );

    initial begin
        // 显示输出
        $monitor("t=%0t : a=%b b=%b sel=%b -> y=%b", $time, a, b, sel, y);

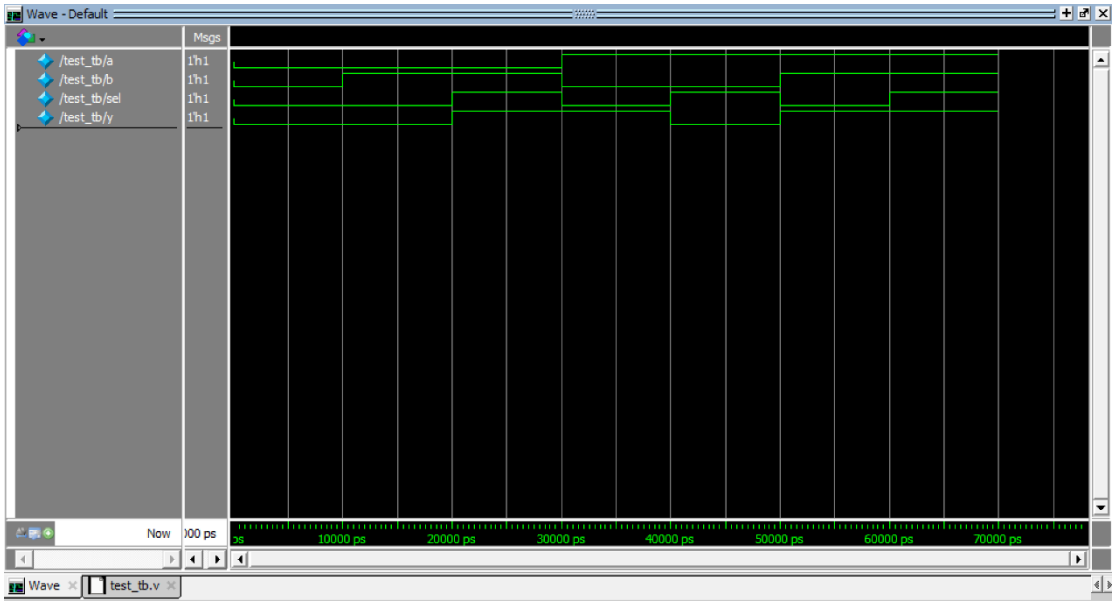
        // 初始化
        a   = 0;
        b   = 0;
        sel = 0;
```

```
        // 测试各种组合
        #10 a=0; b=1; sel=0;
        #10 a=0; b=1; sel=1;
        #10 a=1; b=0; sel=0;
        #10 a=1; b=0; sel=1;
        #10 a=1; b=1; sel=0;
        #10 a=1; b=1; sel=1;

        #10 $finish;
    end

endmodule
```

运行后得到波形图：



波形图展示了一个 2 选 1 多路选择器的仿真结果，其中 a 和 b 为输入信号，sel 为选择信号，y 为输出信号。图中显示，当 sel 为低电平时，输出 y 跟随输入 a；当 sel 为高电平时，输出 y 跟随输入 b。这与 Verilog 代码中定义的逻辑一致，验证了多路选择器按预期工作。

2. 设计一款可综合的 2-4 译码器，并编写 testbench 测试

模块源码：

```
module test(
    input       [1:0]   in,
    output      [3:0]   out
);
assign out[0] = ~in[1] & ~in[0];
assign out[1] = ~in[1] &  in[0];
assign out[2] =  in[1] & ~in[0];
assign out[3] =  in[1] &  in[0];
```

```
endmodule
```

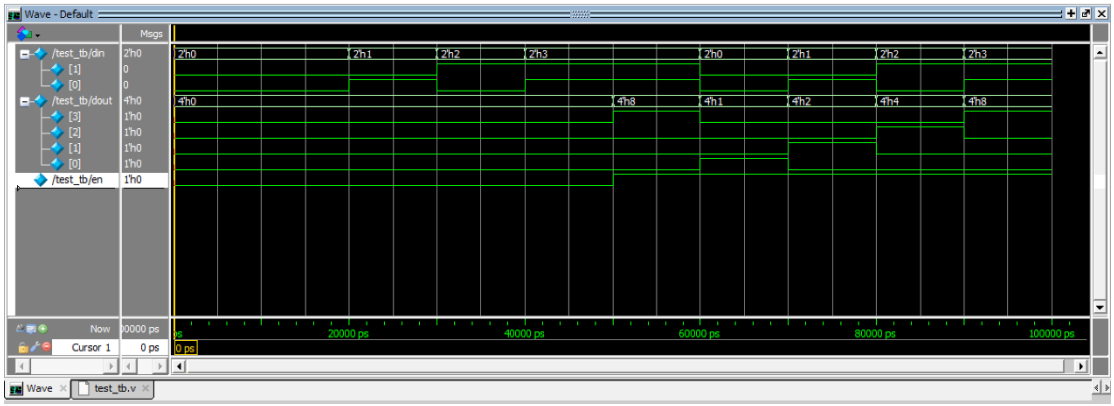
测试模块：

```
`timescale 1ns/1ps
module tb_test();
    reg     [1:0]     in_test;
    wire    [3:0]     out_test;

    initial begin
                      in_test = 2'b00;
        #20           in_test = 2'b01;
        #20           in_test = 2'b10;
        #20           in_test = 2'b11;
    end

    test(.in(in_test),
                      .out(out_test)
    );
endmodule
```

运行后得到波形图：



波形图显示了一个 2 位输入信号/test_tb/din 和其对应的 4 位输出信号/test_tb/dout。输入信号在不同的时间段内取值从 2'h0 到 2'h3，即二进制的 00 到 11。输出信号/test_tb/dout 则根据输入信号的值，显示了相应的 4 位二进制数，每个数位对应一个输入组合。例如，当输入为 2'h0（即 00）时，输出为 4'h0（即 0000），当输入为 2'h1（即 01）时，输出为 4'h1（即 0001），依此类推。这表明输出信号正确地反映了输入信号的组合，符合 2 到 4 线译码器的预期行为。

3．设计一款可综合的 8-3 编码器，并编写 testbench 测试

模块源码：

```
module test(
    input     [7:0]     in,
    output    [2:0]     out
);
    assign out[2] = in[7] | in[6] | in[5] | in[4];
    assign out[1] = in[7] | in[6] | in[3] | in[2];
    assign out[0] = in[7] | in[5] | in[3] | in[1];
```

```
endmodule
```

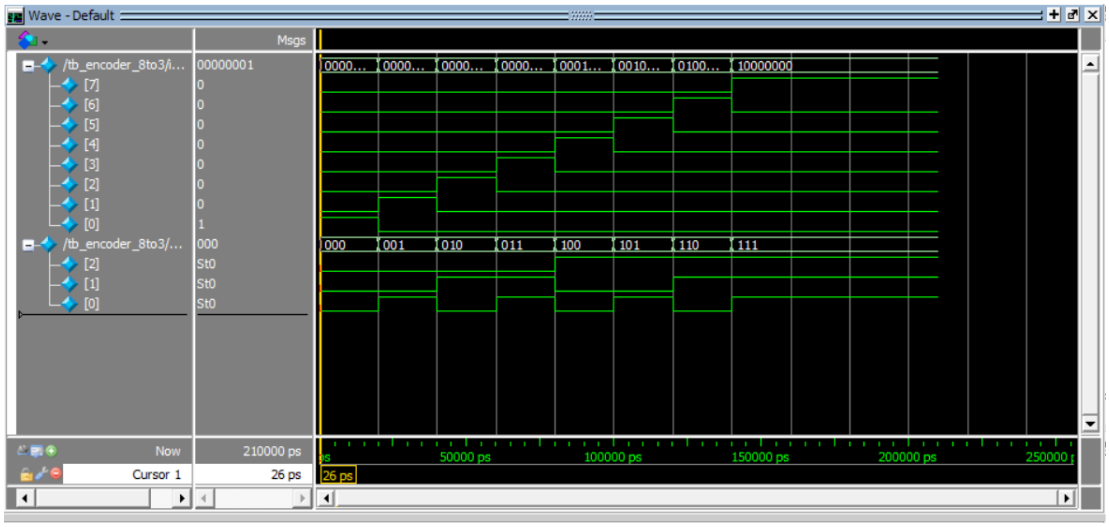
测试模块：

```
`timescale 1ns/1ps
module tb_test ();
    reg     [7:0]     in_test;
    wire    [2:0]     out_test;

    initial begin
                in_test = 8'b0000_0001;
        #20     in_test = 8'b0000_0010;
        #20     in_test = 8'b0000_0100;
        #20     in_test = 8'b0000_1000;
        #20     in_test = 8'b0001_0000;
        #20     in_test = 8'b0010_0000;
        #20     in_test = 8'b0100_0000;
        #20     in_test = 8'b1000_0000;
    end

    test test(.in(in_test),
                    .out(out_test)
    );
endmodule
```

运行后得到波形图：



波形图展示了一个 8 位输入信号 in 和其对应的 3 位输出信号 out。该模块是一个简单的逻辑电路，其中输出 out 的每一位由输入 in 的特定位通过或逻辑（OR）组合而成。具体来说，out[2]由 in[7]到 in[4]中的任一位为 1 时输出 1，out[1]由 in[7]、in[6]、in[3]、in[2]中的任一位为 1 时输出 1，out[0]由 in[7]、in[5]、in[3]、in[1]中的任一位为 1 时输出 1。波形图显示了输入信号在不同时间点的值以及对应的输出信号，验证了逻辑电路按预期工作，即当输入信号的相应位中任意一位为高电平时，输出信号的对应位为高电平。

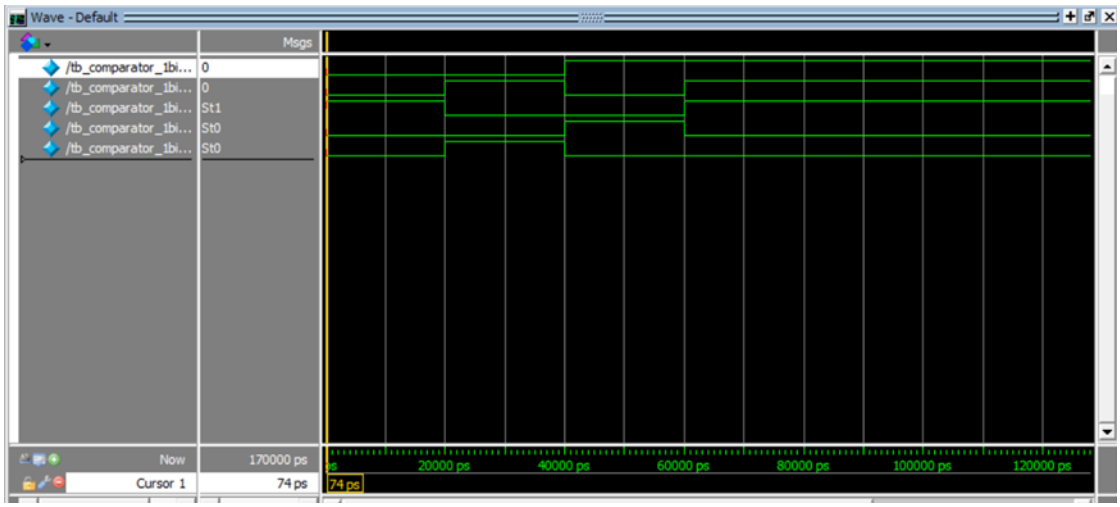
4. 设计一款可综合的 1 位二进制比较器，并编写 testbench 测试

模块源码：
```
module test(
        input a,
        input b,
        output eq,
        output gt,
        output lt
);
    assign eq = ~(a ^ b);
    assign gt = a & ~b;
    assign lt = ~a & b;
endmodule
```
测试模块：
```
`timescale 1ns/1ps
module tb_test();
    reg a_test;
    reg b_test;
    wire eq_test;
    wire gt_test;
    wire lt_test;

    initial begin
            {a_test, b_test}    = 2'b00;
        #20 {a_test, b_test}    = 2'b01;
        #20 {a_test, b_test}    = 2'b10;
        #20 {a_test, b_test}    = 2'b11;
    end

    test test(.a(a_test),
              .b(b_test),
              .eq(eq_test),
              .gt(gt_test),
              .lt(lt_test)
    );
Endmodule
```

运行后得到波形图：



波形图展示了一个简单的比较器模块，该模块接收两个输入信号 a 和 b，并输出三个信号 eq（相等）、gt（大于）和 lt（小于）。根据 Verilog 代码，eq 信号在 a 和 b 相等时为高电平，gt 信号在 a 大于 b 时为高电平，lt 信号在 a 小于 b 时为高电平。波形图中，a 和 b 的值在不同时间段内变化，导致 eq、gt 和 lt 信号相应变化。例如，当 a 和 b 同时为高或同时为低时，eq 为高；当 a 为高而 b 为低时，gt 为高；当 a 为低而 b 为高时，lt 为高。波形图验证了比较器逻辑的正确性。

5．设计一款可综合的 2+2 位简单全加器，并编写 testbench 测试

模块源码：

```
module full_adder(
        input a,
        input b,
        input cin,
        output sum,
        output cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b | cin);
endmodule


module test(
        input   [1:0]   a,
        input   [1:0]   b,
        input           cin0,
        output  [1:0]   sum,
        output          cout
);
    wire cin1;

    full_adder fa0(.a(a[0]),
                   .b(b[0]),
                   .cin(cin0),
```

```
                .sum(sum[0]),
                .cout(cin1)
        );
        full_adder fa1(.a(a[1]),
                .b(b[1]),
                .cin(cin1),
                .sum(sum[1]),
                .cout(cout)
        );
    endmodule
```

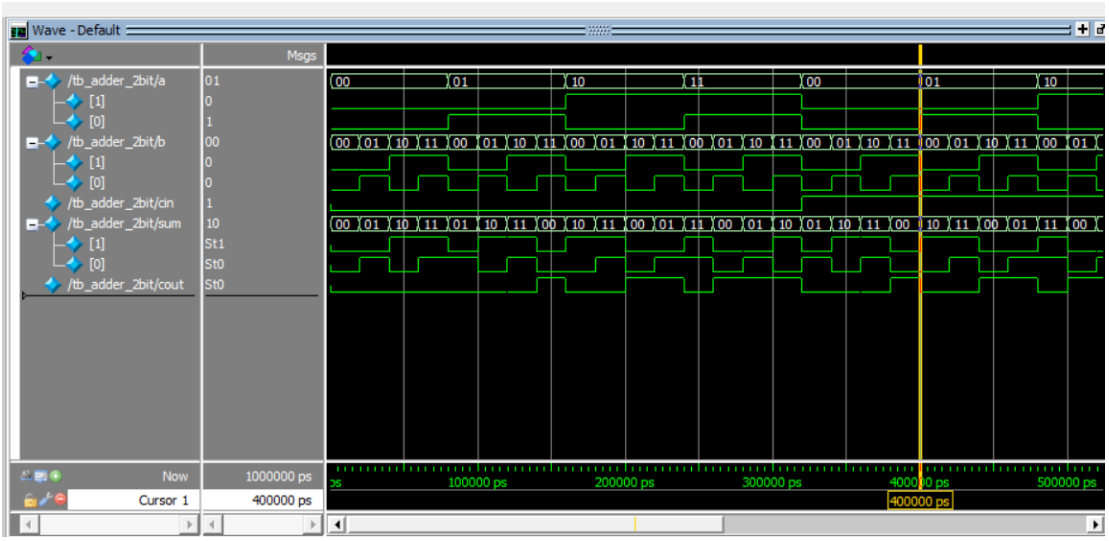
测试源码：

```
`timescale 1ns/1ps
    module tb_test ();
        reg     [1:0]   a;
        reg     [1:0]   b;
        reg             cin;
        wire    [1:0]   sum;
        wire            cout;

        integer i;
        initial begin
            for (i = 0; i < 32; i = i + 1) begin
                {cin, a, b} = i;
                #20;
            end
        end

        test test(
            .a(a),
            .b(b),
            .cin0(cin),
            .sum(sum),
            .cout(cout)
        );
    endmodule
```

运行后得到波形图：



波形图展示了一个 2 位全加器模块的仿真结果。该模块由两个 full_adder 实例组成，分别处理位 a[0]和 a[1]的加法运算。输入信号 a 和 b 各为 2 位，cin0 为初始进位输入。波形图中，a 和 b 的值在不同时间段内变化，导致 sum 和 cout 信号相应变化。例如，当 a[0]和 b[0]为 1 且 cin0 为 1 时，sum[0]为 0 且产生进位 cin1。随后，cin1 作为进位输入与 a[1]和 b[1]一起被加到第二个 full_adder 中，影响 sum[1]和最终的进位输出 cout。波形图验证了全加器模块按预期工作，正确处理了多位二进制数的加法运算。

六、实验心得

通过本次实验，我对 Verilog 语言在数字电路设计中的应用有了更深刻的理解和实践经验。实验中，我设计并测试了多种组合逻辑电路，包括 2 选 1 多路选择器、2-4 译码器、8-3 编码器、1 位二进制比较器和 2+2 位全加器。这些设计不仅加深了我对 Verilog 语法和结构的理解，也让我体会到了硬件描述语言与高级编程语言在设计思想和实现方式上的差异。在编写 testbench 进行仿真测试的过程中，我学会了如何系统地验证电路的功能正确性。通过观察波形图，我能够直观地看到输入信号与输出信号之间的关系，这对于调试和优化设计非常有帮助。此外，实验中的自学环节让我认识到了 Verilog 编程规范的重要性，良好的编程习惯可以提高代码的可读性和可维护性，同时也有助于减少资源消耗和提高设计效率。

总的来说，这次实验不仅提升了我的 Verilog 编程技能，也增强了我对数字电路设计的理解。未来，我将继续深入学习 EDA 工具的使用，提高自己的设计能力，为解决更复杂的电路设计问题打下坚实的基础。